

小橙项目知识库 — 原理性知识全面讲解

本文档系统性整理小橙项目涉及的所有原理性知识，涵盖硬件、电子、通信协议、软件架构等方面。

适合从零开始学习，建议按章节顺序阅读。

目录

- [1. 硬件平台基础](#)
- [2. 电源系统](#)
- [3. 电机驱动原理](#)
- [4. PWM 脉宽调制](#)
- [5. I2C 总线通信](#)
- [6. ADC 模数转换 \(ADS1115\)](#)
- [7. 分压电路与电压监控](#)
- [8. SPI 总线与 WS2812B 灯带](#)
- [9. PCA9685 PWM 扩展芯片](#)
- [10. MOSFET 功率开关 \(IRF520\)](#)
- [11. 摄像头与视频流](#)
- [12. 音频系统](#)
- [13. Python asyncio 异步编程](#)
- [14. FastAPI Web 框架](#)
- [15. WebSocket 实时通信](#)
- [16. Vue 3 前端框架](#)
- [17. 软件架构设计模式](#)
- [18. 信号滤波与数据平滑](#)
- [19. 差速转向算法](#)
- [20. 安全系统设计](#)

知识库 Part 1 — 硬件平台 + 电源系统 + 电机驱动 + PWM

1. 硬件平台基础

1.1 Orange Pi 5 Pro (RK3588S)

Orange Pi 5 Pro 是一款基于 Rockchip RK3588S 芯片的单板计算机 (SBC)，类似树莓派但性能更强。

核心规格：

项目	参数
CPU	4× Cortex-A76 (大核 2.4GHz) + 4× Cortex-A55 (小核 1.8GHz)
NPU	6TOPS, 支持 INT4/INT8/INT16/FP16
RAM	8GB LPDDR4X
存储	TF 卡 (本项目用 64GB A2 级)
GPIO	40-pin 扩展口, 兼容树莓派引脚布局
接口	USB 3.0 × 2、USB 2.0 × 2、HDMI × 2、PCIe、I2C、SPI、UART
功耗	典型 5-10W, 峰值约 15W

为什么选它： RK3588S 自带 6TOPS NPU，Phase 5 的 YOLO 推理和 Phase 13 的本地 LLM 都可以不依赖云端。40-pin GPIO 满足全部 14 个 Phase 的外设需求。

1.2 40-pin GPIO 引脚系统

Opi 5 Pro 的 40-pin 引脚有三套编号系统，容易混淆：

编号系统	说明	使用场景
物理引脚 (Physical)	1-40，按板子上的实际位置编号	接线时对照图纸
wiringPi 编号 (wPi)	wiringOP 库使用的逻辑编号	代码中 <code>wiringpi.pinMode(wPi, ...)</code>
BCM 编号	树莓派 BCM 编号，Opi 部分兼容	一般不用

本项目用 **wiringPi 编号**，因为驱动层用 wiringOP-Python。

1.3 wiringOP-Python

wiringOP 是 Orange Pi 官方维护的 GPIO 库，是树莓派 wiringPi 的移植版。

关键 API：

```
import wiringpi
from wiringpi import GPIO

wiringpi.wiringPiSetup()           # 初始化（必须第一步）
wiringpi.pinMode(pin, GPIO.OUTPUT) # 设置引脚模式
wiringpi.digitalWrite(pin, GPIO.HIGH) # 写高电平
wiringpi.digitalRead(pin)          # 读引脚
```

安装方式（必须从源码编译）：

```
git clone https://github.com/orangepi-xunlong/wiringOP.git -b next
cd wiringOP && sudo ./build
cd python && sudo python3 setup.py install
```

不能 `pip install wiringpi`，那是旧版，不支持 RK3588S。

1.4 sysfs 接口

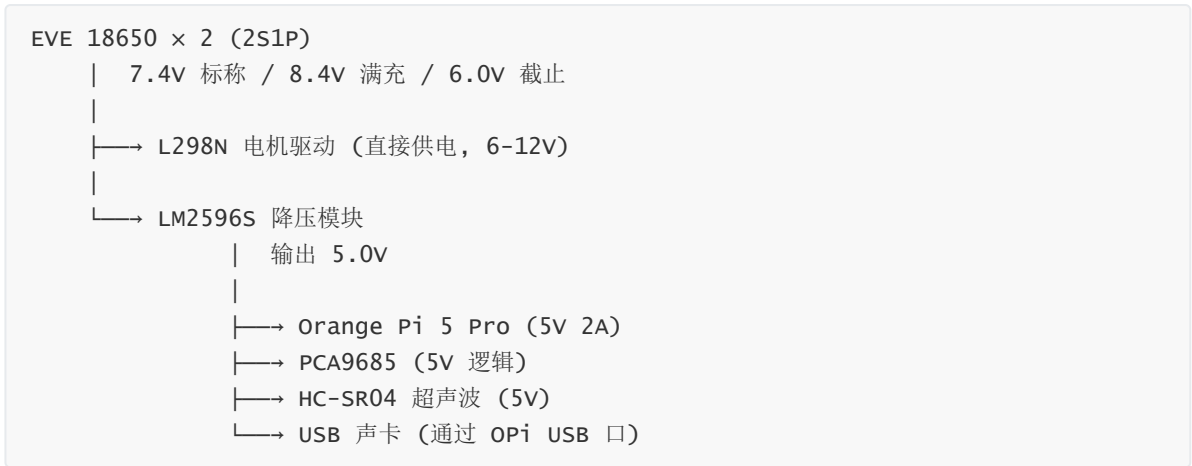
Linux 内核通过 `/sys` 文件系统暴露硬件控制接口，读写文件即可控制硬件。

<code>/sys/class/pwm/pwmchip2/</code>	← PWM 控制器
<code>/sys/class/gpio/</code>	← GPIO 控制
<code>/sys/class/thermal/</code>	← 温度传感器
<code>/proc/net/wireless</code>	← WiFi 信号强度
<code>/proc/stat</code>	← CPU 使用率

本项目的 PWM 电机控制完全通过 sysfs 实现，不依赖 wiringOP 的 PWM 功能（wiringOP PWM 在 RK3588S 上有兼容性问题）。

2. 电源系统

2.1 整体电源拓扑



2.2 18650 锂电池 (2S1P)

2S1P 含义：

- 2S = 2 节串联 (Series)，电压叠加： $3.7V \times 2 = 7.4V$ 标称
- 1P = 1 并联 (Parallel)，容量不变

电压区间：

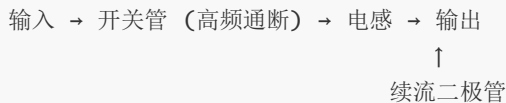
状态	单节电压	总电压
满充	4.2V	8.4V
标称	3.7V	7.4V
低压告警	3.6V	7.2V
截止	3.0V	6.0V

为什么要监控电压： LM2596S 降压模块需要约 7V 最低输入才能稳定输出 5V。电池低于 7V 时，OPi 可能无声断电，导致 TF 卡文件系统损坏。这是本项目接 ADS1115 的核心原因。

2.3 LM2596S 降压模块

LM2596S 是一款开关型降压 (Buck) 稳压芯片，效率远高于线性稳压器 (LDO)。

工作原理 (Buck 变换器)：



通过调节开关管的占空比控制输出电压。占空比 = 输出/输入，例如 8V 输入、5V 输出，占空比约 62.5%。

关键参数：

- 输入范围：4.5V - 40V
- 输出范围：1.25V - 37V (通过电位器调节)
- 最大电流：3A
- 效率：约 75-92% (远优于 LDO 的 $(V_{out}/V_{in}) \times 100\%$)
- **最低输入约 7V 才能稳定输出 5V (压差约 2V)**

调压方法： 顺时针旋转电位器升压，逆时针降压。用万用表测输出端，调到 5.0V。

2.4 18650 保护板 (3A 过充保护)

保护板集成在电池组中，提供：

- **过充保护：** 单节超过 4.25V 时切断充电
- **过放保护：** 单节低于 2.5V 时切断放电
- **过流保护：** 电流超过 3A 时切断

注意： 保护板的过流限制是 3A，L298N 电机堵转时电流可能超过此值，导致保护板跳闸。实际使用中电机不会长时间堵转，问题不大。

2.5 电量百分比计算

不能简单线性映射，因为锂电池放电曲线是非线性的：

```
def voltage_to_percent(voltage: float) -> int:
    # 分段线性近似锂电池放电曲线
    if voltage >= 8.4: return 100
    if voltage >= 8.0: return int(80 + (voltage - 8.0) / 0.4 * 20)
    if voltage >= 7.6: return int(50 + (voltage - 7.6) / 0.4 * 30)
    if voltage >= 7.2: return int(20 + (voltage - 7.2) / 0.4 * 30)
    if voltage >= 6.8: return int(5 + (voltage - 6.8) / 0.4 * 15)
    return 0
```

中间段 (7.6-8.0V) 电压变化缓慢，对应大部分使用时间；两端变化较快。

3. 电机驱动原理

3.1 直流电机基础

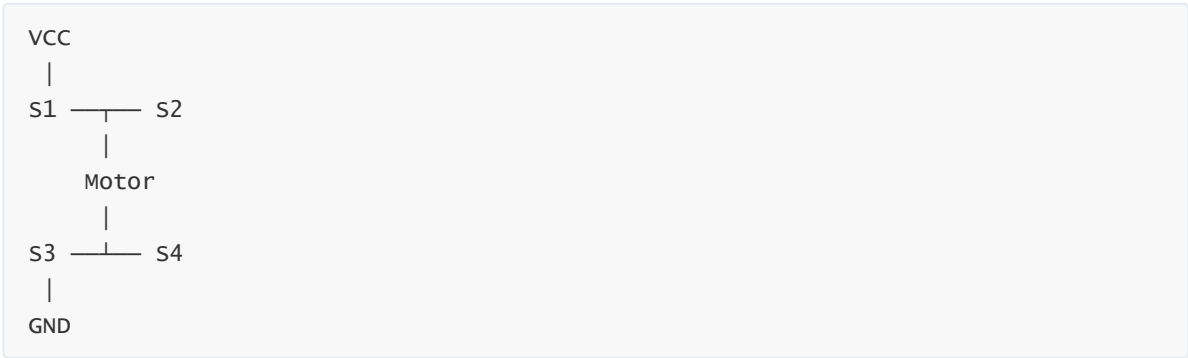
直流电机通过电磁感应将电能转化为机械能。

转速控制： 转速与电压成正比。降低电压 → 降低转速。实际中用 PWM 模拟不同电压。

转向控制： 改变电流方向 → 改变磁场方向 → 改变转向。H 桥电路实现这一功能。

3.2 H 桥电路

H 桥是控制直流电机正反转的标准电路，因形状像字母 H 而得名。



S1	S2	S3	S4	效果
ON	OFF	OFF	ON	正转
OFF	ON	ON	OFF	反转
ON	OFF	ON	OFF	制动 (两端接 VCC)
OFF	ON	OFF	ON	制动 (两端接 GND)
OFF	OFF	OFF	OFF	滑行停止

制动 vs 滑行： 制动时两端短路，产生反向电动势阻止转动，停车更快。滑行时电机自由转动直到摩擦停止。

3.3 L298N 电机驱动模块

L298N 是一款集成双 H 桥的电机驱动芯片，可同时控制两路电机。

引脚说明：

引脚	功能
IN1, IN2	左电机方向控制
IN3, IN4	右电机方向控制
ENA	左电机使能 (PWM 调速)
ENB	右电机使能 (PWM 调速)
OUT1-4	电机输出
VCC	电机电源 (6-12V)
5V	逻辑电源 (可由内部稳压提供)

方向控制逻辑：

IN1	IN2	效果
HIGH	LOW	正转
LOW	HIGH	反转
HIGH	HIGH	制动
LOW	LOW	停止

关键坑：ENA/ENB 跳线帽

L298N 出厂时 ENA/ENB 通过跳线帽短接到 5V（固定全速使能）。接 PWM 调速前**必须先拔掉跳线帽**，否则 PWM 信号被覆盖，电机始终全速。

3.4 死区 (Dead Zone)

电机存在最低启动电压，低于此电压时电机不转但有电流（发热）。

本项目设 `MOTOR_DEAD_ZONE = 40`，意味着：

- 用户输入 0-100% 的速度
- 实际输出 40-100% 的 PWM 占空比
- 低于 40% 的 PWM 电机不转，直接输出 0

```
def _map_speed(self, speed: float) -> float:
    if speed <= 0:
        return 0
    dz = config.MOTOR_DEAD_ZONE # 40
    return dz + speed * (100 - dz) / 100
    # 输入 50% → 输出 40 + 50*(60/100) = 70%
    # 输入 100% → 输出 40 + 100*(60/100) = 100%
```

4. PWM 脉宽调制

4.1 PWM 基本原理

PWM (Pulse Width Modulation，脉宽调制) 通过快速开关信号模拟模拟量输出。

关键参数：

- **频率 (Frequency)**：每秒开关多少次，单位 Hz。本项目 1kHz（周期 1ms）
- **占空比 (Duty Cycle)**：高电平时间占一个周期的比例，0-100%
- **等效电压**：占空比 × 电源电压。50% 占空比 ≈ 2.5V（5V 系统）

100% 占空比：  （全速）

50% 占空比：  （半速）

0% 占空比：  （停止）

为什么 1kHz：太低 (<100Hz) 电机抖动，能听到嗡嗡声；太高 (>20kHz) 超出 L298N 响应速度。1kHz 是电机驱动的常用频率。

4.2 RK3588S sysfs PWM

RK3588S 通过 Linux sysfs 接口控制硬件 PWM，操作方式是读写文件：

```
# 1. 导出 PWM 通道
echo 0 > /sys/class/pwm/pwmchip2/export

# 2. 设置周期（纳秒）
echo 1000000 > /sys/class/pwm/pwmchip2/pwm0/period # 1ms = 1kHz

# 3. 设置占空比（纳秒，必须 ≤ period）
echo 500000 > /sys/class/pwm/pwmchip2/pwm0/duty_cycle # 50%

# 4. 设置极性
echo normal > /sys/class/pwm/pwmchip2/pwm0/polarity
# 或
echo inversed > /sys/class/pwm/pwmchip2/pwm0/polarity

# 5. 使能
echo 1 > /sys/class/pwm/pwmchip2/pwm0/enable
```

Python 代码中用 `open(path, 'w').write(str(value))` 实现同样操作。

4.3 PWM 极性反转 (RK3588S 特有坑)

现象：写入占空比 100% 时电机反而最慢，写入 0% 时全速。

根因：RK3588S 的 sysfs PWM 默认极性与预期相反。写入 `duty_cycle = x` 实际高电平时间是 `period - x`。

解决：设置 `polarity = inversed`，或在代码中反转计算：

```
PWM_INVERTED = True # config.py

def set_duty(self, percent: float):
    actual = (100 - percent) if self.inverted else percent
    duty_ns = int(self.period_ns * actual / 100)
    self._write(f"{self.pwm_path}/duty_cycle", duty_ns)
```

4.4 PWM 通道与 pwmchip 编号

RK3588S 有多个 PWM 控制器，每个对应一个 pwmchip：

PWM 通道	pwmchip	物理引脚	用途
PWM13_M2	pwmchip2	7	左电机 ENA
PWM14_M2	pwmchip3	32	右电机 ENB

如何确认 pwmchip 编号：

```
ls /sys/class/pwm/
# 输出: pwmchip0  pwmchip1  pwmchip2  pwmchip3 ...

# 查看每个 chip 对应哪个 PWM 控制器
cat /sys/class/pwm/pwmchip2/device/uevent
```

4.5 PCA9685 的 PWM (I2C 扩展)

除了 sysfs 硬件 PWM，本项目还用 PCA9685 芯片通过 I2C 扩展出 16 路 PWM，用于舵机和大灯控制。详见第 9 章。

两种 PWM 的对比：

特性	sysfs 硬件 PWM	PCA9685 I2C PWM
通道数	受限 (RK3588S 约 4 路可用)	16 路
精度	纳秒级	12-bit (4096 级)
CPU 占用	零 (硬件生成)	极低 (I2C 写寄存器)
用途	电机调速 (需要高精度)	舵机、LED 调光

知识库 Part 2 — I2C + ADC + 分压电路 + SPI/WS2812B + PCA9685 + MOSFET

5. I2C 总线通信

5.1 I2C 基本原理

I2C (Inter-Integrated Circuit) 是一种两线串行通信协议，由飞利浦（现 NXP）发明。

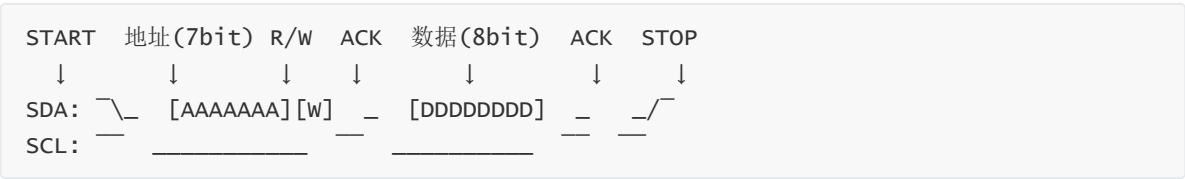
两根线：

- **SDA (Serial Data)**：数据线，双向
- **SCL (Serial Clock)**：时钟线，由主设备控制

特点：

- 一主多从：一个主设备可挂多个从设备，通过 7-bit 地址区分
- 半双工：同一时刻只能单向传输
- 速度：标准 100kHz，快速 400kHz，高速 3.4MHz
- 需要上拉电阻（通常 4.7kΩ）将总线拉到高电平

5.2 I2C 通信时序



- **START**：SCL 高时 SDA 下降沿
- **STOP**：SCL 高时 SDA 上升沿

- **ACK**: 从设备拉低 SDA 表示收到
- **NACK**: SDA 保持高表示未收到或结束

5.3 OPi 5 Pro 的 I2C 总线

RK3588S 有多个 I2C 控制器，本项目使用 **I2C1_M4**:

总线	物理引脚	设备文件	挂载设备
I2C1_M4	3 (SDA), 5 (SCL)	/dev/i2c-1	ADS1115 (0x48) + PCA9685 (0x40)

检测 I2C 设备:

```
i2cdetect -y 1
# 输出矩阵中 40 = PCA9685, 48 = ADS1115
```

5.4 原始 I2C 操作 (Python)

本项目不用 smbus2 等第三方库，直接用 Python 的 `ioctl` 操作 I2C:

```
import os
import fcntl
import struct

I2C_SLAVE = 0x0703

fd = os.open("/dev/i2c-1", os.O_RDWR)
fcntl.ioctl(fd, I2C_SLAVE, 0x48) # 选择从设备地址

# 写寄存器
os.write(fd, bytes([reg_addr, value_high, value_low]))

# 读数据
os.write(fd, bytes([reg_addr])) # 先写寄存器地址
data = os.read(fd, 2)          # 再读 2 字节
```

为什么不用 smbus2: 零第三方依赖，代码透明，便于理解底层原理，也避免了依赖版本问题。

6. ADC 模数转换 (ADS1115)

6.1 为什么需要 ADC

OPi 5 Pro 没有内置 ADC（模数转换器）。GPIO 引脚只能读数字信号（高/低），无法直接读取电池电压这样的模拟量。

ADS1115 是一款通过 I2C 通信的外置 16-bit ADC，解决了这个问题。

6.2 ADS1115 规格

参数	值
分辨率	16-bit (实际 15-bit + 符号位)
通道数	4 路单端 (A0-A3) 或 2 路差分
接口	I2C, 地址 0x48-0x4B (由 ADDR 引脚决定)
采样率	8-860 SPS (可配置)
满量程 (FSR)	$\pm 0.256V$ 到 $\pm 6.144V$ (6 档可配置)
工作电压	2.0V - 5.5V

地址配置：

ADDR 引脚	I2C 地址
GND	0x48 (本项目)
VCC	0x49
SDA	0x4A
SCL	0x4B

6.3 ADS1115 寄存器

ADS1115 有 4 个 16-bit 寄存器：

寄存器	地址	功能
Conversion	0x00	转换结果 (只读)
Config	0x01	配置 (量程/通道/采样率等)
Lo_thresh	0x02	低阈值 (比较器模式用)
Hi_thresh	0x03	高阈值 (比较器模式用)

Config 寄存器关键位 (16-bit)：

Bit 15:	OS	- 单次转换触发 (写1启动)
Bit 14-12:	MUX	- 通道选择 (100=A0 vs GND)
Bit 11-9:	PGA	- 增益/量程 (001= $\pm 4.096V$)
Bit 8:	MODE	- 0=连续, 1=单次
Bit 7-5:	DR	- 采样率 (100=128SPS)

6.4 读取流程

```
# 1. 写 Config 寄存器, 触发单次转换
config_val = 0xC183 # OS=1(启动), MUX=100(A0), PGA=001( $\pm 4.096V$ ), MODE=1(单次)
os.write(fd, struct.pack('>BH', 0x01, config_val))

# 2. 等待转换完成 (约 8ms @ 128SPS)
time.sleep(0.01)
```

```
# 3. 读 Conversion 寄存器
os.write(fd, bytes([0x00]))
raw = os.read(fd, 2)
value = struct.unpack('>h', raw)[0] # 有符号 16-bit

# 4. 转换为电压
voltage = value * 4.096 / 32767.0 # FSR=4.096V, 满量程 32767
```

6.5 16-bit 精度意味着什么

FSR = $\pm 4.096\text{V}$, 分辨率 = $4.096 / 32767 \approx 0.125\text{mV/LSB}$

对于 8V 电池电压（经分压后约 2.64V），精度约 0.125mV，换算回原始电压约 0.38mV。远超实际需求（告警精度 0.1V 级别即可）。

7. 分压电路与电压监控

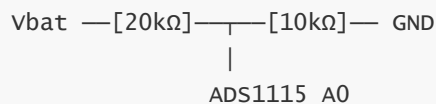
7.1 分压电路原理

ADS1115 的 FSR 最大 $\pm 4.096\text{V}$ ，但电池电压最高 8.4V，超出量程。需要分压电路将电压缩小到可测范围。

分压公式：

$$V_{\text{out}} = V_{\text{in}} \times R_2 / (R_1 + R_2)$$

本项目：R1 = 20k Ω , R2 = 10k Ω



理论分压比：(20+10)/10 = **3.0**

实测校准：万用表测得总压 7.99V，ADS1115 读数 2.64V，实际比值 = 7.99/2.64 = **3.026**（已写入 config.BATTERY_DIVIDER_RATIO）

还原公式：

$$\text{battery_voltage} = \text{adc_voltage} * \text{BATTERY_DIVIDER_RATIO}$$

7.2 为什么用 20k Ω + 10k Ω

- **阻值不能太小**：分压电路持续消耗电流，R 越小电流越大。20k Ω +10k Ω 时电流 = $8.4\text{V}/30\text{k}\Omega \approx 0.28\text{mA}$ ，功耗约 2.4mW，可接受。
- **阻值不能太大**：太大时 ADS1115 输入阻抗影响读数精度（ADS1115 输入阻抗约 10M Ω ，远大于 10k Ω ，影响可忽略）。
- **分压后电压**：8.4V/3 = 2.8V，在 ADS1115 的 $\pm 4.096\text{V}$ 量程内，有足够余量。

7.3 EMA 滤波（指数移动平均）

ADC 读数有噪声，直接显示会抖动。用 EMA 平滑：

```
# EMA: Exponential Moving Average
# alpha 越小越平滑，但响应越慢
alpha = 0.01 # config.BATTERY_EMA_ALPHA

ema = alpha * new_reading + (1 - alpha) * ema
```

直觉理解： $\alpha=0.01$ 时，新读数只占 1%，历史值占 99%。约需 100 次采样（100 秒）才能完全响应一个阶跃变化。对于缓慢变化的电池电压，这是合适的。

启动初始化： 冷启动时 EMA 初始值为 0，需要多次采样才能收敛。本项目取前 8 次读数的均值作为初始值：

```
samples = [read_adc() for _ in range(BATTERY_EMA_INIT_SAMPLES)]
ema = sum(samples) / len(samples)
```

7.4 电量百分比变化速率限制

即使 EMA 后，电量百分比仍可能因噪声出现短暂跳变。加入变化速率限制：

```
# 每秒最多下降 2%，最多上升 0.5%
max_drop = BATTERY_PERCENT_DROP_RATE * dt # 2 * 1.0 = 2%/s
max_rise = BATTERY_PERCENT_RISE_RATE * dt # 0.5 * 1.0 = 0.5%/s

delta = new_percent - current_percent
delta = max(-max_drop, min(max_rise, delta))
current_percent += delta
```

上升速率限制更严（0.5%/s），防止噪声导致电量虚假回升。

8. SPI 总线与 WS2812B 灯带

8.1 SPI 基本原理

SPI (Serial Peripheral Interface) 是一种四线同步串行通信协议，速度比 I2C 快得多。

四根线：

- **MOSI** (Master Out Slave In)：主发从收
- **MISO** (Master In Slave Out)：从发主收
- **SCLK** (Serial Clock)：时钟
- **CS/SS** (Chip Select)：片选，低有效

特点：

- 全双工（同时收发）
- 无地址，用 CS 选择设备
- 速度可达几十 MHz
- WS2812B 只用 MOSI（单线协议，借用 SPI 时序）

8.2 WS2812B 灯珠原理

WS2812B 是一种集成了控制芯片的 RGB LED，每颗灯珠内置 IC，通过单线串行协议控制。

数据格式： 每颗灯珠接收 24-bit 数据（GRB 顺序，注意不是 RGB）：

```
[G7 G6 G5 G4 G3 G2 G1 G0] [R7 R6 R5 R4 R3 R2 R1 R0] [B7 B6 B5 B4 B3 B2 B1 B0]
```

时序要求（极严格）：

- 逻辑 1：高电平 0.8μs + 低电平 0.45μs
- 逻辑 0：高电平 0.4μs + 低电平 0.85μs
- 复位：低电平 > 50μs

这个时序精度要求在 Python 中无法用 GPIO 直接实现（Python 太慢），需要借用 SPI 硬件。

8.3 SPI Bitbang 模拟 WS2812B 时序

核心思路： 用 SPI 的 MOSI 信号模拟 WS2812B 的单线时序。

SPI 时钟设为 6.4MHz，每个 SPI bit 周期 = 1/6.4MHz ≈ 156ns。

用 3 个 SPI bit 表示 1 个 WS2812B bit：

- WS2812B 逻辑 1 → SPI 发送 110（高高低）≈ 312ns 高 + 156ns 低
- WS2812B 逻辑 0 → SPI 发送 100（高低低）≈ 156ns 高 + 312ns 低

```
def _encode_byte(byte_val: int) -> list[int]:
    """将 1 字节编码为 WS2812B SPI 序列"""
    result = []
    for bit in range(7, -1, -1):
        if byte_val & (1 << bit):
            result.extend([0b110]) # 逻辑 1
        else:
            result.extend([0b100]) # 逻辑 0
    return result
```

每颗灯珠 24-bit → 72 个 SPI bit → 9 字节 SPI 数据。10 颗灯珠 = 90 字节。

8.4 灯带分段控制

本项目 10 颗灯珠按位置分为三段：

```
[0][1][2]   [3][4][5][6]   [7][8][9]
左侧       尾灯中央       右侧
```

```
STRIP_SEGMENTS = {
    "left": (0, 2), # 索引 0-2，共 3 颗
    "tail": (3, 6), # 索引 3-6，共 4 颗
    "right": (7, 9), # 索引 7-9，共 3 颗
}
```

`set_segment("tail", 255, 0, 0)` 只点亮尾灯段为红色，其余不变。

8.5 电平转换（5V 信号问题）

WS2812B 数据线需要 5V 逻辑电平，而 OPi 的 SPI MOSI 是 3.3V。

解决方案： 74AHCT125 单向电平转换芯片（3.3V → 5V），或直接用 3.3V 信号（部分 WS2812B 可接受，但不保证稳定）。

9. PCA9685 PWM 扩展芯片

9.1 为什么需要 PCA9685

RK3588S 的硬件 PWM 通道有限（可用的约 4 路），而本项目需要：

- 2 路大灯 PWM（左/右独立调光）
- 2 路舵机 PWM（pan/tilt）

PCA9685 通过 I2C 扩展出 16 路 12-bit PWM，一颗芯片解决所有需求。

9.2 PCA9685 规格

参数	值
PWM 通道	16 路
分辨率	12-bit (4096 级)
频率范围	24Hz - 1526Hz（可配置）
接口	I2C，地址 0x40-0x7F
工作电压	2.3V - 5.5V
输出电流	每路最大 25mA（直接驱动 LED），驱动电机/大灯需外接 MOSFET

9.3 PWM 频率设置

PCA9685 通过预分频寄存器设置 PWM 频率：

```
# 内部振荡器 25MHz
# prescale = round(25MHz / (4096 * freq)) - 1
prescale = round(25_000_000 / (4096 * freq)) - 1

# 写入前需先进入 SLEEP 模式
write_reg(0x00, 0x10)      # MODE1: SLEEP=1
write_reg(0xFE, prescale)  # PRE_SCALE
write_reg(0x00, 0x00)      # MODE1: 唤醒
time.sleep(0.005)
write_reg(0x00, 0xA0)      # MODE1: AUTO_INCREMENT=1
```

9.4 通道控制寄存器

每个通道有 4 个字节的寄存器（ON_L, ON_H, OFF_L, OFF_H）：

ON = 上升沿时刻 (0-4095)
OFF = 下降沿时刻 (0-4095)
占空比 = (OFF - ON) / 4096

通常 ON=0, 只设 OFF 值:

```
def set_pwm(channel: int, on: int, off: int):  
    base = 0x06 + channel * 4  
    write_reg(base, on & 0xFF)  
    write_reg(base + 1, on >> 8)  
    write_reg(base + 2, off & 0xFF)  
    write_reg(base + 3, off >> 8)  
  
# 50% 占空比  
set_pwm(0, 0, 2048)  
  
# 全亮 (特殊值)  
set_pwm(0, 0, 4096) # OFF bit 12 = 1 表示全开  
  
# 全灭  
set_pwm(0, 0, 0)
```

9.5 舵机控制

SG90 舵机通过 PWM 控制角度:

- 频率: 50Hz (周期 20ms)
- 脉宽 0.5ms (占空比 2.5%) → 0°
- 脉宽 1.5ms (占空比 7.5%) → 90° (中位)
- 脉宽 2.5ms (占空比 12.5%) → 180°

```
def angle_to_pwm(angle: float, freq: int = 50) -> int:  
    """角度转 PCA9685 OFF 值"""  
    # 脉宽范围 0.5ms - 2.5ms  
    pulse_ms = 0.5 + (angle / 180.0) * 2.0  
    # 转换为 4096 级  
    period_ms = 1000.0 / freq # 20ms  
    return int(pulse_ms / period_ms * 4096)
```

9.6 通道分配 (本项目)

PCA9685 通道	用途	频率
0	左大灯 (IRF520 MOSFET)	1kHz
1	右大灯 (IRF520 MOSFET)	1kHz
2	云台水平舵机 (pan)	50Hz
3	云台垂直舵机 (tilt)	50Hz

注意: 大灯和舵机频率不同, 但 PCA9685 所有通道共用同一个频率设置。解决方案: 大灯用 PWM 占空比调光 (1kHz 对 MOSFET 没问题), 舵机用 50Hz。实际上 50Hz 对大灯调光也可以, 只是频率低时肉眼可能看到闪烁。本项目选择 50Hz 以兼容舵机。

10. MOSFET 功率开关 (IRF520)

10.1 为什么需要 MOSFET

PCA9685 每路输出最大 25mA，而 3W LED 在 5V 下电流约 600mA，远超 PCA9685 的驱动能力。

IRF520 是一款 N 沟道 MOSFET，作为功率开关，用 PCA9685 的小信号控制大电流负载。

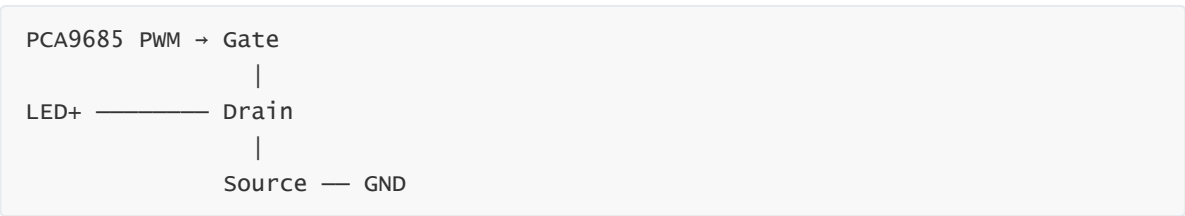
10.2 MOSFET 工作原理

MOSFET 有三个引脚：

- **Gate (G)**：控制极，输入电压控制开关
- **Drain (D)**：漏极，电流从这里流入
- **Source (S)**：源极，电流从这里流出

N 沟道 MOSFET：

- V_{gs} (Gate-Source 电压) > 阈值电压 (约 2-4V) → 导通 (开关闭合)
- V_{gs} < 阈值电压 → 截止 (开关断开)



10.3 IRF520 参数

参数	值
类型	N 沟道增强型 MOSFET
Vds (最大漏源电压)	100V
Id (最大漏极电流)	9.7A
Vgs(th) (阈值电压)	2-4V
Rds(on) (导通电阻)	0.27Ω

为什么选 IRF520： 阈值电压 2-4V，PCA9685 输出 3.3V 可以驱动（勉强，实测可行）。更好的选择是 IRLZ44N（逻辑电平 MOSFET，阈值 1-2V，3.3V 完全驱动）。

10.4 PWM 调光原理

MOSFET 在 PWM 信号下快速开关，LED 的平均亮度 = 占空比 × 最大亮度。

占空比 100%	→ LED 全亮
占空比 50%	→ LED 半亮（视觉上约 70% 亮度，因为人眼对亮度感知是非线性的）
占空比 0%	→ LED 熄灭

频率选择： 1kHz 对 MOSFET 开关损耗可忽略，且远超人眼临界融合频率（约 60Hz），不会看到闪烁。

知识库 Part 3 — 摄像头/视频流 + 音频系统 + asyncio + FastAPI

11. 摄像头与视频流

11.1 OV5640 摄像头

OV5640 是一款 500 万像素 CMOS 图像传感器，通过 USB UVC 协议连接到 OPI。

UVC (USB Video Class): Linux 内核原生支持的 USB 摄像头标准，无需额外驱动，插上即用，设备文件为 `/dev/video0`（或 `video1`、`video2` 等）。

关键参数：

- 分辨率：最高 2592×1944，本项目用 1280×720
- 帧率：720P 下约 12-30fps（取决于光线和 USB 带宽）
- 格式：MJPG（硬件压缩 JPEG）或 YUYV（原始 YUV）

11.2 OpenCV 采集

```
import cv2

cap = cv2.VideoCapture(0) # 打开 /dev/video0
cap.set(cv2.CAP_PROP_FRAME_WIDTH, 1280)
cap.set(cv2.CAP_PROP_FRAME_HEIGHT, 720)
cap.set(cv2.CAP_PROP_FPS, 30)

# 使用 MJPG 格式（硬件压缩，大幅降低 CPU 占用）
cap.set(cv2.CAP_PROP_FOURCC, cv2.VideoWriter_fourcc(*'MJPG'))

ret, frame = cap.read() # frame 是 numpy array (H, W, 3) BGR
```

MJPG vs YUYV:

- YUYV：原始 YUV 格式，每帧 720P = 1280×720×2 ≈ 1.8MB，USB 带宽压力大，CPU 需要解码
- MJPG：摄像头内部 JPEG 压缩，每帧约 50-200KB，USB 带宽小，CPU 只需解压

本项目用 MJPG，实测 CPU 占用从 ~40% 降到 ~5%。

11.3 MJPEG 流媒体

MJPEG (Motion JPEG) 是最简单的视频流格式：连续发送 JPEG 图片，用 HTTP multipart 协议分隔。

HTTP multipart 格式：

```
HTTP/1.1 200 OK
Content-Type: multipart/x-mixed-replace; boundary=frame

--frame
Content-Type: image/jpeg
Content-Length: 12345

[JPEG 二进制数据]
--frame
Content-Type: image/jpeg
...
```

浏览器的 `` 原生支持这种格式，会持续更新显示最新帧。

FastAPI 实现：

```
from fastapi.responses import StreamingResponse

async def generate():
    while True:
        frame = vision.get_latest_frame()
        if frame is not None:
            yield (b'--frame\r\n'
                  b'Content-Type: image/jpeg\r\n\r\n'
                  + frame + b'\r\n')
            await asyncio.sleep(1/30) # 30fps

@router.get("/stream/camera")
async def camera_stream():
    return StreamingResponse(
        generate(),
        media_type="multipart/x-mixed-replace; boundary=frame"
    )
```

11.4 采集线程架构

摄像头采集是阻塞操作（`cap.read()` 会等待下一帧），不能在 `asyncio` 主循环中直接调用。

解决方案： 独立线程 + 共享帧缓冲

```
import threading

class VisionSubsystem:
    def __init__(self):
        self._frame = None # 最新帧 (JPEG bytes)
        self._lock = threading.Lock()
        self._thread = None
        self._running = False

    def _capture_loop(self):
        cap = cv2.VideoCapture(0)
        while self._running:
            ret, frame = cap.read()
            if ret:
                _, jpeg = cv2.imencode('.jpg', frame, [cv2.IMWRITE_JPEG_QUALITY,
80])
```

```

        with self._lock:
            self._frame = jpeg.tobytes()

    def get_latest_frame(self):
        with self._lock:
            return self._frame

```

为什么用线程而不是 asyncio: `cap.read()` 是同步阻塞调用, 无法 `await`。用 `run_in_executor` 也可以, 但独立线程更简洁, 且摄像头采集是持续高频操作, 线程更合适。

11.5 延迟分析

摄像头曝光	→ USB 传输	→ OpenCV 解码	→ JPEG 编码	→ HTTP 发送	→ 浏览器解码	→ 显示
~33ms	~5ms	~2ms	~5ms	~网络	~5ms	

局域网总延迟约 50-100ms, 对 FPV 遥控可接受 (专业 FPV 要求 <50ms, 需要 WebRTC)。

12. 音频系统

12.1 USB 声卡 (Jieli UACDemoV1.0)

USB 免驱声卡通过 USB Audio Class (UAC) 协议工作, Linux 内核原生支持。

查看声卡:

```

aplay -l
# card 3: UACDemoV10 [UACDemoV1.0], device 0: USB Audio [USB Audio]

```

关键参数:

- 卡号: 3 (`AUDIO_CARD = 3`)
- 采样率: 48000Hz
- 格式: S16LE (16-bit 小端有符号整数)

12.2 aplay 播放 WAV

```

aplay -D hw:3,0 audio.wav
# -D hw:3,0 指定声卡 3 设备 0

```

Python 中用 subprocess 调用:

```

import subprocess
proc = subprocess.Popen(
    ["aplay", "-D", f"hw:{AUDIO_CARD},0", filepath],
    stdout=subprocess.DEVNULL,
    stderr=subprocess.DEVNULL,
)
await asyncio.get_event_loop().run_in_executor(None, proc.wait)

```

12.3 ffplay 播放 MP3

aplay 只支持 WAV，MP3 需要 ffplay (FFmpeg 的播放器)：

```
ffplay -nodisp -autoexit -loglevel quiet \  
-af "volume=2.0" \  
-i audio.mp3
```

-nodisp：不显示视频窗口

-autoexit：播放完自动退出

-af "volume=2.0"：音量放大 2 倍

12.4 edge-tts 文字转语音

edge-tts 是微软 Edge 浏览器 TTS 引擎的 Python 客户端，免费，中文效果好。

```
import edge_tts  
import asyncio  
  
async def tts(text: str, output_file: str):  
    communicate = edge_tts.Communicate(  
        text,  
        voice="zh-CN-YunxiNeural" # 中文男声  
    )  
    await communicate.save(output_file)  
  
# 生成后用 ffplay 播放
```

可用中文声音：

- zh-CN-YunxiNeural：男声（本项目默认）
- zh-CN-XiaoxiaoNeural：女声
- zh-CN-YunjianNeural：男声（更成熟）

12.5 amixer 音量控制

amixer 是 ALSA 的命令行混音器：

```
# 查看控件  
amixer -c 3 controls  
  
# 设置音量（通过 numid）  
amixer -c 3 cset numid=4 80%  
  
# 查询当前音量  
amixer -c 3 cget numid=4
```

Python 中：

```
def set_volume(level: int): # level: 0-100
    raw = int(level / 100 * AUDIO_VOLUME_MAX) # 映射到 0-147
    subprocess.run(
        ["amixer", "-c", str(AUDIO_CARD), "cset", f"numid={AUDIO_VOLUME_NUMID}",
        str(raw)],
        capture_output=True
    )
```

12.6 多通道音频管理

本项目同时可能有多个音频来源（鸣笛、倒车提示、TTS、告警），需要管理优先级：

通道	用途	优先级
horn	鸣笛（按住循环）	最高，压制其他
tts	文字转语音	高，暂停倒车提示
alert	低电量告警	中
reverse	倒车提示音	低，可被其他打断
sfx	普通音效	最低

实现方式： 每个通道独立 subprocess，需要打断时 `proc.kill()`。

13. Python asyncio 异步编程

13.1 为什么需要异步

小橙后端需要同时处理：

- WebSocket 消息（高频，每 100ms 一条）
- 遥测推送（10Hz motion + 1Hz sensors）
- 摄像头流（30fps）
- 安全看门狗（100ms 轮询）
- 避障扫描（100ms 轮询）

如果用同步代码，这些任务会互相阻塞。asyncio 让单线程可以并发处理多个 I/O 任务。

13.2 事件循环与协程

```
import asyncio

# 协程函数（用 async def 定义）
async def say_hello():
    print("Hello")
    await asyncio.sleep(1) # 让出控制权，不阻塞事件循环
    print("World")

# 运行
asyncio.run(say_hello())
```

关键概念：

- **事件循环 (Event Loop)**: 单线程调度器, 轮流执行各协程
- **协程 (Coroutine)**: 可暂停/恢复的函数, `await` 处暂停
- **Task**: 包装协程, 让事件循环调度它
- **await**: 暂停当前协程, 让事件循环运行其他任务

13.3 并发执行多个任务

```
# 方式 1: create_task (立即调度, 不等待)
task = asyncio.create_task(some_coroutine())

# 方式 2: gather (等待所有完成)
results = await asyncio.gather(
    coroutine_a(),
    coroutine_b(),
    coroutine_c(),
)

# 方式 3: 后台任务 (fire and forget)
asyncio.create_task(background_loop())
```

本项目的遥测推送:

```
async def run(self):
    self._running = True
    await asyncio.gather(
        self._publish_motion(), # 10Hz
        self._publish_sensors(), # 1Hz
    )
```

13.4 阻塞操作的处理

asyncio 是单线程的, 任何阻塞调用都会卡住整个事件循环。

规则: 阻塞操作必须放到线程池执行:

```
loop = asyncio.get_event_loop()

# I2C 读取 (阻塞)
result = await loop.run_in_executor(None, blocking_i2c_read)

# 超声波测距 (阻塞, 需要等待 echo)
distance = await loop.run_in_executor(None, driver.measure, "front")
```

`run_in_executor(None, func, *args)` 将 `func(*args)` 放到默认线程池执行, 返回 awaitable。

13.5 asyncio 与 FastAPI 的集成

FastAPI 基于 asyncio, 路由处理函数可以是协程:

```
@app.websocket("/ws/control")
async def websocket_endpoint(websocket: WebSocket):
    await websocket.accept()
    while True:
        data = await websocket.receive_json() # 等待消息，不阻塞
        result = await dispatcher.dispatch(data)
        if result:
            await websocket.send_json(result)
```

lifespan 中启动后台任务：

```
@asynccontextmanager
async def lifespan(app: FastAPI):
    # 启动
    safety_task = asyncio.create_task(safety.run())
    yield
    # 关闭
    safety_task.cancel()
```

13.6 asyncio 常见陷阱

陷阱 1: 忘记 await

```
# 错误：协程对象没有被执行
result = some_coroutine() # 只创建了协程对象

# 正确
result = await some_coroutine()
```

陷阱 2: 在协程中调用阻塞函数

```
# 错误：time.sleep 阻塞整个事件循环
async def bad():
    time.sleep(1)

# 正确
async def good():
    await asyncio.sleep(1)
```

陷阱 3: Task 异常被吞掉

```
# create_task 的异常不会自动传播
task = asyncio.create_task(risky_coroutine())
# 如果 risky_coroutine 抛异常，会被静默忽略

# 解决：添加异常回调
task.add_done_callback(lambda t: t.exception() and logger.error(t.exception()))
```

14. FastAPI Web 框架

14.1 FastAPI 基础

FastAPI 是基于 Starlette 和 Pydantic 的现代 Python Web 框架，原生支持 asyncio。

核心特性：

- 自动生成 OpenAPI 文档（访问 `/docs`）
- 类型提示驱动的请求/响应验证
- 原生 WebSocket 支持
- 与 asyncio 生态无缝集成

14.2 路由注册

```
from fastapi import FastAPI, APIRouter

app = FastAPI()
router = APIRouter(prefix="/api")

@router.get("/status")
async def get_status():
    return {"status": "ok"}

app.include_router(router)
```

本项目将路由分散到三个模块：

- `api/http.py`：HTTP REST 接口（状态查询、配置）
- `api/websocket.py`：WebSocket 控制通道
- `api/stream.py`：MJPEG 视频流

14.3 lifespan 生命周期管理

FastAPI 的 lifespan 替代了旧的 `startup` / `shutdown` 事件：

```
from contextlib import asynccontextmanager

@asynccontextmanager
async def lifespan(app: FastAPI):
    # 启动阶段：初始化所有子系统
    motion.init()
    sensing.init()
    safety_task = asyncio.create_task(safety.run())

    yield # 应用运行中

    # 关闭阶段：清理资源
    safety_task.cancel()
    motion.cleanup()

app = FastAPI(lifespan=lifespan)
```

为什么用 lifespan： 确保资源在应用退出时正确释放（GPIO 复位、PWM 关闭等），避免下次启动时硬件状态异常。

14.4 CORS 中间件

开发时前端 (localhost:5173) 和后端 (localhost:8000) 端口不同，浏览器会拒绝跨域请求。

```
from fastapi.middleware.cors import CORSMiddleware

app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],      # 生产环境应限制为具体域名
    allow_methods=["*"],
    allow_headers=["*"],
)
```

14.5 静态文件服务

生产部署时，Vue 构建产物可以由 FastAPI 直接服务：

```
from fastapi.staticfiles import StaticFiles

# 挂载到根路径，html=True 表示 SPA 模式（所有路径返回 index.html）
app.mount("/", StaticFiles(directory="frontend/dist", html=True),
name="frontend")
```

注意：静态文件挂载必须在所有路由注册之后，否则会拦截 API 请求。

14.6 依赖注入模式

本项目用简单的模块级全局变量 + `init()` 函数注入依赖，而不是 FastAPI 的 `Depends()`：

```
# api/websocket.py
_dispatcher: Dispatcher = None
_safety: SafetyWatchdog = None

def init(dispatcher, safety, telemetry):
    global _dispatcher, _safety, _telemetry
    _dispatcher = dispatcher
    _safety = safety
    _telemetry = telemetry
```

这种方式更简单，适合单实例应用。FastAPI 的 `Depends()` 更适合需要请求级别作用域的场景（如数据库连接）。

知识库 Part 4 — WebSocket + Vue 3 + 软件架构 + 信号滤波 + 差速算法 + 安全系统

15. WebSocket 实时通信

15.1 WebSocket vs HTTP

特性	HTTP	WebSocket
连接模式	请求-响应，每次新连接	持久连接，全双工
延迟	每次请求有握手开销	建立后几乎零开销
服务端推送	不支持（需轮询）	原生支持
适用场景	低频查询、文件传输	实时控制、遥测推送

小橙的控制指令需要 100ms 级别的实时性，遥测需要服务端主动推送，WebSocket 是唯一合理选择。

15.2 WebSocket 握手

WebSocket 基于 HTTP 升级协议：

```
客户端 → 服务端：
GET /ws/control HTTP/1.1
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Key: dGhlIHNhbXBsZSBub25jZQ==

服务端 → 客户端：
HTTP/1.1 101 Switching Protocols
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Accept: s3pPLMBiTxaQ9kYGzzhZRbK+x0o=
```

握手完成后，连接升级为 WebSocket，双方可以随时发送消息。

15.3 Envelope 协议设计

本项目所有 WS 消息使用统一的 Envelope 格式：

```
{
  "type": "cmd.motion",
  "id": "optional-uuid-for-ack",
  "ts": 1744876800.123,
  "payload": {
    "vx": 0.5,
    "vy": 0.8
  }
}
```

type 前缀规则：

- `cmd.*`：前端 → 后端，控制指令
- `tel.*`：后端 → 前端，遥测数据
- `event.*`：双向，事件通知

为什么用 Envelope： 随着功能增加，消息类型从 2 种增长到 20+。统一格式让前端可以用一个 switch 路由所有消息，新增功能只加新 type，协议层零改动。

15.4 FastAPI WebSocket 实现

```
from fastapi import WebSocket, WebSocketDisconnect

@router.websocket("/ws/control")
async def websocket_endpoint(websocket: WebSocket):
    await websocket.accept()
    safety.touch()

    # 启动遥测推送
    telemetry.set_send_fn(websocket.send_json)
    tel_task = asyncio.create_task(telemetry.run())

    try:
        while True:
            data = await websocket.receive_json()
            safety.touch() # 喂狗

            result = await dispatcher.dispatch(data)
            if result:
                await websocket.send_json(result)

    except WebSocketDisconnect:
        safety.on_disconnect()
    finally:
        tel_task.cancel()
        telemetry.set_send_fn(None)
```

15.5 前端 WebSocket (Vue)

```
// composables/useWebSocket.ts
const ws = new WebSocket('ws://192.168.0.x:8000/ws/control')

ws.onopen = () => { connected.value = true }
ws.onclose = () => { connected.value = false }

ws.onmessage = (event) => {
    const msg = JSON.parse(event.data)
    switch (msg.type) {
        case 'tel.motion':
            carStore.updateMotion(msg.payload)
            break
        case 'tel.sensors':
            carStore.updateSensors(msg.payload)
            break
        case 'event.pong':
            // 计算 ws 延迟
            latency.value = Date.now() - pingTime
            break
    }
}

// 发送指令
function send(type: string, payload: object) {
    ws.send(JSON.stringify({ type, ts: Date.now() / 1000, payload }))
}
```

15.6 持续运动策略

摇杆拖拽时，前端每 100ms 重发当前 vx/vy:

```
// 摇杆移动时
onJoystickMove(vx: number, vy: number) {
  this.currentVx = vx
  this.currentVy = vy
}

// 定时器每 100ms 发送
setInterval(() => {
  if (connected.value) {
    send('cmd.motion', { vx: currentVx, vy: currentVy })
  }
}, 100)
```

为什么需要重发：安全看门狗超时 500ms 停车。如果只在摇杆变化时发送，摇杆静止时 500ms 后会误停车。持续重发既保活看门狗，又让后端始终知道最新摇杆位置。

15.7 WS 延迟测量 (ping/pong)

```
// 前端每 5 秒发一次 ping
setInterval(() => {
  pingTime = Date.now()
  send('cmd.ping', {})
}, 5000)

// 收到 pong 时计算延迟
case 'event.pong':
  wsLatency.value = Date.now() - pingTime
  break
```

后端立即回复:

```
def _handle_ping(self, payload: dict) -> dict:
    return {
        "type": "event.pong",
        "ts": time.time(),
        "payload": {},
    }
```

16. Vue 3 前端框架

16.1 Vue 3 核心概念

Composition API (组合式 API)：Vue 3 的主要编程模型，用 `setup()` 函数或 `<script setup>` 语法组织逻辑。

```
<script setup lang="ts">
import { ref, computed, onMounted } from 'vue'
```

```

const count = ref(0) // 响应式基本类型
const doubled = computed(() => count.value * 2) // 计算属性

onMounted(() => {
  console.log('组件挂载完成')
})
</script>

<template>
  <button @click="count++">{{ count }} (doubled: {{ doubled }})</button>
</template>

```

16.2 Pinia 状态管理

Pinia 是 Vue 3 官方推荐的状态管理库，替代 Vuex。

```

// stores/carStore.ts
import { defineStore } from 'pinia'

export const useCarStore = defineStore('car', {
  state: () => ({
    connected: false,
    battery: { voltage: 0, percent: 0, level: 'unknown' },
    motion: { vx: 0, vy: 0, speed: 0, direction: 'idle' },
    sensors: { cpu_temp: 0, wifi_rssi: null },
  }),

  actions: {
    updateMotion(payload: any) {
      this.motion = { ...this.motion, ...payload }
    },
    updateSensors(payload: any) {
      this.sensors = { ...this.sensors, ...payload }
      if (payload.voltage) this.battery.voltage = payload.voltage
    },
  },
})

```

在组件中使用：

```

<script setup>
import { useCarStore } from '@/stores/carStore'
const store = useCarStore()
</script>

<template>
  <div>电量: {{ store.battery.percent }}%</div>
</template>

```

16.3 虚拟摇杆实现

虚拟摇杆通过 touch/mouse 事件计算偏移量，映射到 vx/vy：

```

function onTouchMove(event: TouchEvent) {
  const touch = event.touches[0]

```

```

const rect = joystickEl.getBoundingClientRect()
const centerX = rect.left + rect.width / 2
const centerY = rect.top + rect.height / 2

let dx = touch.clientX - centerX
let dy = touch.clientY - centerY

// 限制在圆形范围内
const maxRadius = rect.width / 2
const dist = Math.sqrt(dx * dx + dy * dy)
if (dist > maxRadius) {
  dx = dx / dist * maxRadius
  dy = dy / dist * maxRadius
}

// 归一化到 [-1, 1]
vx.value = dx / maxRadius
vy.value = -dy / maxRadius // Y 轴反转（屏幕向下为正，但前进为正）
}

```

16.4 Vite 构建工具

Vite 是现代前端构建工具，开发时用原生 ES Module，构建时用 Rollup。

开发代理配置（`vite.config.ts`）：

```

export default defineConfig({
  server: {
    proxy: {
      '/ws': {
        target: 'ws://192.168.0.x:8000',
        ws: true,
      },
      '/stream': {
        target: 'http://192.168.0.x:8000',
      },
      '/api': {
        target: 'http://192.168.0.x:8000',
      },
    },
  },
})

```

开发时前端跑在 localhost:5173，通过代理转发到 OPi 后端，避免 CORS 问题。

17. 软件架构设计模式

17.1 六层架构

前端层 (Vue 3) ← 人机交互, 无业务逻辑
↓ WebSocket / HTTP / MJPEG
接口层 (FastAPI) ← 协议解析, envelope 路由
↓ Python 函数调用
业务层 (Dispatcher/Safety) ← 编排、安全、遥测
↓ Python 函数调用
子系统层 (motion/lighting/...) ← 领域语义, 不感知 GPIO
↓ Protocol 接口
驱动层 (motor/adc/camera/...) ← 硬件封装, Real+Mock 双实现
↓ sysfs/I2C/SPI/USB
硬件层 (电机/传感器/摄像头/...)

核心原则：每层只依赖其直接下层，禁止跨层调用。

17.2 Protocol 模式（接口隔离）

每个驱动定义一个 Protocol (Python 的结构化子类型)，Real 和 Mock 都实现同一接口：

```
# drivers/motor/protocol.py
from typing import Protocol

class MotorDriverProtocol(Protocol):
    def init(self) -> None: ...
    def set_motors(self, left: float, right: float) -> None: ...
    def stop(self) -> None: ...
    def brake(self) -> None: ...
    def cleanup(self) -> None: ...
    @property
    def current_state(self) -> dict: ...
```

```
# drivers/motor/__init__.py
from app import config

if config.USE MOCK:
    from .mock import MockMotorDriver as MotorDriver
else:
    from .real import RealMotorDriver as MotorDriver

__all__ = ["MotorDriver"]
```

好处：

- 子系统层只依赖 Protocol，不关心是 Real 还是 Mock
- 换硬件只改驱动层，上层代码不动
- Mock 实现强制与 Real 保持接口一致

17.3 Dispatcher 模式（命令路由）

Dispatcher 是一个命令路由表，将 WS 消息 type 映射到处理函数：

```
self._handlers = {
    "cmd.motion": self._handle_motion,
    "cmd.brake": self._handle_brake,
    "cmd.gimbal": self._handle_gimbal,
    "cmd.light": self._handle_light,
```

```

"cmd.audio": self._handle_audio,
"cmd.nitro": self._handle_nitro,
"cmd.obstacle": self._handle_obstacle,
}

async def dispatch(self, message: dict):
    handler = self._handlers.get(message["type"])
    if handler:
        return await handler(message["payload"])

```

新增功能只需：写新 handler → 注册到 `_handlers` → 完成。核心 dispatch 逻辑不动。

17.4 观察者模式（回调注入）

子系统间的事件通知用回调函数，避免直接依赖：

```

# obstacle 子系统不直接调用 safety，而是通过回调
class ObstacleSubsystem:
    def set_callbacks(self, on_front_blocked=None, on_rear_blocked=None):
        self._on_front_blocked = on_front_blocked
        self._on_rear_blocked = on_rear_blocked

    def _check_distance(self, distance):
        if distance < STOP_DISTANCE:
            if self._on_front_blocked:
                self._on_front_blocked() # 调用注入的回调

# main.py 中注入
safety.set_obstacle(obstacle) # safety 内部注册回调

```

好处：obstacle 不需要 import safety，两者解耦。

17.5 依赖注入（构造函数注入）

子系统的外部依赖通过构造函数或 setter 注入，而不是在内部创建：

```

# 好的做法：依赖从外部注入
class NitroSubsystem:
    def set_dependencies(self, motion=None, lighting=None, audio=None):
        self._motion = motion
        self._lighting = lighting
        self._audio = audio

# main.py 中组装
nitro.set_dependencies(motion=motion, lighting=lighting, audio=audio)

```

好处：测试时可以注入 Mock 对象，不需要真实硬件。

18. 信号滤波与数据平滑

18.1 为什么需要滤波

传感器读数包含噪声：

- ADC 量化噪声（最低有效位抖动）
- 电源纹波干扰
- 电磁干扰（电机 PWM 对 I2C 的干扰）

直接显示原始数据会导致数值跳动，影响用户体验和决策准确性。

18.2 EMA（指数移动平均）

EMA 是最简单高效的实时滤波算法：

```
class EMFilter:
    def __init__(self, alpha: float, init_value: float = 0.0):
        self.alpha = alpha      # 平滑系数, 0 < alpha < 1
        self.value = init_value

    def update(self, new_sample: float) -> float:
        self.value = self.alpha * new_sample + (1 - self.alpha) * self.value
        return self.value
```

alpha 选择指南：

alpha	特性	适用场景
0.01	极平滑，响应慢（~100 次收敛）	电池电压（变化极慢）
0.1	较平滑，响应中等（~10 次收敛）	温度、距离
0.5	轻微平滑，响应快（~3 次收敛）	需要快速响应的信号

EMA vs 移动平均（SMA）：

- SMA 需要存储 N 个历史值，内存占用随窗口大小增加
- EMA 只需存储一个值，内存 O(1)
- EMA 对近期数据权重更高，响应更自然

18.3 变化速率限制（Rate Limiting）

防止数值突变（噪声尖峰或传感器故障）：

```
def rate_limited_update(current: float, target: float,
                        max_rate: float, dt: float) -> float:
    """
    current: 当前值
    target: 目标值
    max_rate: 每秒最大变化量
    dt: 时间步长（秒）
    """
    max_delta = max_rate * dt
    delta = target - current
    delta = max(-max_delta, min(max_delta, delta))
    return current + delta
```

本项目电量百分比：下降最快 2%/s，上升最快 0.5%/s。

18.4 超声波测距滤波

HC-SR04 偶尔会返回异常值（0 或极大值）：

```
def measure_filtered(self, direction: str) -> float | None:
    readings = []
    for _ in range(3): # 取 3 次读数
        d = self._raw_measure(direction)
        if d is not None and 2 < d < 400: # 有效范围 2-400cm
            readings.append(d)
        time.sleep(0.01)

    if not readings:
        return None
    return sorted(readings)[len(readings) // 2] # 取中位数
```

中位数滤波对异常值（outlier）有很好的抑制效果。

19. 差速转向算法

19.1 差速驱动原理

四轮小车（实际上是两驱差速，前后轮同侧联动）通过左右轮速度差实现转向：

- 左右同速 → 直线行驶
- 左快右慢 → 右转
- 左慢右快 → 左转
- 左前右后 → 原地右旋
- 左后右前 → 原地左旋

19.2 Arcade Drive 算法

本项目使用 Arcade Drive（街机驾驶）算法，将摇杆的 vx/vy 映射到左右轮速：

```
def handle_command(self, vx: float, vy: float) -> None:
    # vx: -1(左) ~ +1(右)，控制转向
    # vy: -1(后) ~ +1(前)，控制前后

    left = vy + vx # 左轮 = 前后 + 转向
    right = vy - vx # 右轮 = 前后 - 转向

    # 等比缩放，保持转向比，防止超出 [-1, 1]
    max_val = max(abs(left), abs(right), 1.0)
    left /= max_val
    right /= max_val

    self._driver.set_motors(left * 100, right * 100)
```

直觉验证：

vx	vy	left	right	效果
0	1	1	1	全速前进
0	-1	-1	-1	全速后退
1	0	1	-1	原地右旋
-1	0	-1	1	原地左旋
0.5	0.5	1.0	0.0	右弧线前进（缩放后）
1	1	1.0	0.0	右弧线前进（缩放后，max=2）

19.3 等比缩放的必要性

不缩放时，`vx=1, vy=1` 会得到 `left=2, right=0`，超出电机范围。

简单截断（`left = min(left, 1.0)`）会破坏转向比：

- 截断前：left=2, right=0, 比值 2:0
- 截断后：left=1, right=0, 比值 1:0（相同，OK）
- 但 `vx=0.5, vy=1` → left=1.5, right=0.5, 截断后 left=1, right=0.5, 比值从 3:1 变成 2:1

等比缩放保持比值：

```
max_val = max(abs(left), abs(right), 1.0) # 找最大绝对值
left /= max_val # 等比缩放
right /= max_val
```

19.4 氮气加速的速度叠加

氮气激活时，`vx/vy` 乘以加速倍率，但需要重新钳位到 `[-1, 1]`：

```
if self._nitro and self._nitro.is_active:
    boost = self._nitro.boost_factor # 1.3
    vx = max(-1.0, min(1.0, vx * boost))
    vy = max(-1.0, min(1.0, vy * boost))
```

效果：原来 `vy=0.77` 的速度，乘以 1.3 后 = 1.0（钳位），达到最大速度。对于已经接近最大速度的指令，氮气效果更明显。

20. 安全系统设计

20.1 安全层次

小橙的安全系统分三层：

- 第一层：硬件安全
- 18650 保护板（过充/过放/过流）
 - LM2596S 欠压保护
- 第二层：软件安全（Safetywatchdog）
- WS 断连/超时自动停车
 - 前方障碍物自动停车

- 后方障碍物倒车刹停
- 低电压告警

第三层：用户界面安全

- 刹车按钮优先级最高
- 断连时 UI 显示离线状态
- 电量 HUD 实时显示

20.2 看门狗 (Watchdog) 原理

看门狗是一种超时检测机制：定期"喂狗"（重置计时器），如果超时未喂狗，触发安全动作。

```
class Safetywatchdog:
    def touch(self):
        """喂狗：收到任何 WS 消息时调用"""
        self._last_heartbeat = time.time()
        self._has_connection = True

    async def run(self):
        while self._running:
            if self._has_connection:
                elapsed = time.time() - self._last_heartbeat
                if elapsed > WS_DISCONNECT_TIMEOUT: # 500ms
                    self._motion.stop()
                    self._has_connection = False
            await asyncio.sleep(0.1) # 100ms 轮询
```

超时阈值选择：500ms。前端每 100ms 发一次运动指令，正常情况下看门狗每 100ms 被喂一次。500ms 超时 = 允许连续 5 次丢包，足够应对短暂网络抖动，又不会让失控小车跑太远。

20.3 刹车优先级与保护窗口

刹车指令优先级高于运动指令。刹车后有 300ms 保护窗口，期间忽略运动指令：

```
BRAKE_SUPPRESS_SECONDS = 0.3

def _handle_brake(self, payload):
    self._brake_until = time.monotonic() + BRAKE_SUPPRESS_SECONDS
    self._motion.brake()

def _handle_motion(self, payload):
    if time.monotonic() < self._brake_until and (vx != 0 or vy != 0):
        logger.debug("忽略刹车保护窗口内的运动指令")
        return
    # 正常处理运动指令
```

为什么需要保护窗口：前端发送刹车时，队列中可能已有运动指令在途。没有保护窗口，刹车后立即被运动指令覆盖，刹车无效。

20.4 避障安全联锁

避障是硬联锁（不可绕过），在 dispatcher 层实现：

```
def _handle_motion(self, payload):
    # 前方避障：阻止前进
    if self._obstacle and self._obstacle.front_blocked and vy > 0:
        vy = 0 # 强制清零前进分量，但允许转向

    # 后方避障：阻止倒车
    if self._obstacle and self._obstacle.rear_blocked and vy < 0:
        vy = 0 # 强制清零后退分量
```

设计细节：只清零 vy（前后），保留 vx（转向）。前方有障碍时，用户仍可以原地转向，找到出路。

20.5 低电压告警流程

```
SensingSubsystem 读取电压
↓
voltage < BATTERY_LOW (6.8V)
↓
推送 event.alert 到前端 (TopBar 显示红色警告)
↓
AudioSubsystem.start_low_voltage_alert()
↓
每 10 秒循环播放 "电量不足，请及时充电"
↓
voltage < BATTERY_CRITICAL (6.2V)
↓
强制停车 + 持续告警
```

告警阈值设计：

- `BATTERY_LOW = 6.8V`：开始告警，仍可行驶
- `BATTERY_CRITICAL = 6.2V`：强制停车，防止 LM2596S 欠压导致 OPI 断电

20.6 断连安全回调链

WS 断连时，SafetyWatchdog 触发一系列安全动作：

```
def on_disconnect(self):
    self._motion.stop() # 停车
    self._mode.force_manual() # 强制回手动模式
    self._audio.stop_horn_and_reverse() # 停止鸣笛和倒车提示
    self._lighting.stop_all() # 停止动态灯效（大灯保持）
    self._gimbal.stop_all() # 停止云台移动
    self._nitro.stop_all() # 停止氮气加速
```

大灯为什么不关：断连时小车可能在黑暗环境中，关大灯会让用户找不到小车。安全考虑优先于节能。